

KV Cache as the State Plane of LLM and Multi-Agent Serving

A macro-to-micro survey of the emerging architecture, the forces behind it, the core technical axes, and the institutions shaping the shift.

Thesis. KV cache is moving from engine-internal attention residue into a managed serving asset: stored, routed, compressed, transferred, reused, priced, isolated, and increasingly communicated across models and agents.

0. Visual and reading plan

The report is organized from macro to micro. It first motivates the reader with observable paper/project momentum, then defines scope, then introduces the architectural map, then turns the static map into an evolution story, and only after that compares major components. The final section maps the people and institutions driving the paradigm shift.

Visual	Purpose	Where used
Occurrence timeline by year and branch	Show that KV-cache work became a dense systems topic after 2024, with 2025-2026 branching into production platforms, workload studies, reuse, and multi-agent communication.	Motivation and slide opening.
Landscape architecture map	Give the whole stack: agent runtime, router/scheduler, metadata/compatibility plane, reuse engine, data plane, inference engines.	Architectural overview.
Evolution path diagram	Turn the static architecture into a historical movement: local memory -> prompt reuse -> transport/offload -> phase boundary -> platform -> protocol/workflow state.	Driving forces section.
Break-even decision graph	Clarify when to recompute, exact-reuse, approximate-reuse, transfer, compress, or communicate KV.	Future blueprint.
Comparison matrices	Keep papers comparable only within the same architectural interface: phase disaggregation, retrieval/offload, reuse/communication, compression/retention.	Detailed component sections.
People/institution matrix	Distinguish platform builders, boundary shifters, model-native-state builders, mechanism builders, and product/API builders.	People section.

1. Executive overview

One-sentence summary. The field is evolving from optimizing local KV layout to building a KV-aware serving architecture in which KV is a cross-service state object and, in multi-agent systems, a candidate communication substrate.

The strongest evidence is not a single paper. It is the simultaneity of signals: vLLM/PagedAttention made KV layout a systems abstraction [1]; SGLang and Prompt Cache made repeated prompt structure explicitly reusable [2][3]; Splitwise and DistServe turned the prefill-to-decode handoff into an execution boundary [5][6]; Mooncake made the production platform itself KV-cache-centric [8]; LMCache, Mooncake Transfer Engine, NVIDIA Dynamo/NIXL, and llm-d productize external KV movement and routing [12][40][41][42]; and DroidSpeak, KVCOMM, KVComm, Q-KVComm, and TokenDance push reuse into cross-model and multi-agent communication [17][18][19][20][21].

The core architectural shift is from **token computation throughput** to **token-state serving**. The old serving question was: how many forward passes per second can a model engine run? The new question is: where should the intermediate state of already-computed tokens live, who can reuse it, how precisely must it match, and when is it cheaper to move or repair it than to recompute?

2. Why this suddenly matters: occurrence and institutional momentum

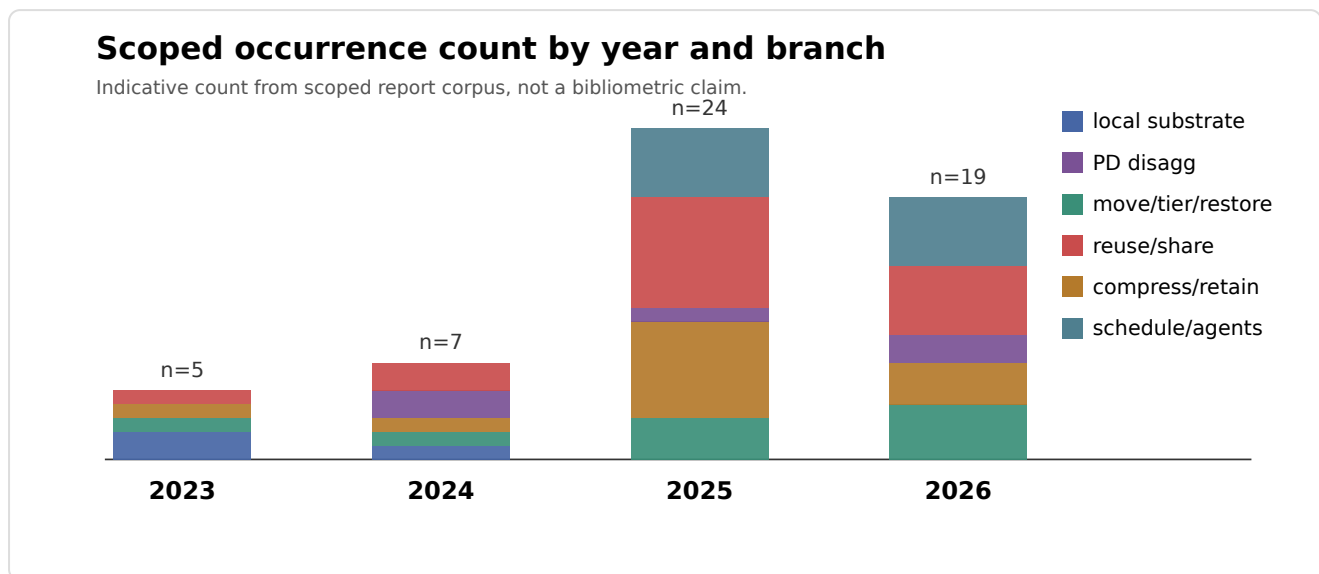


Figure 1. A scoped occurrence view built from this report's corpus. The point is not exact bibliometrics; the point is branching. By 2025-2026, KV-cache work is no longer one line of memory-management papers. It spans systems, storage, networking, scheduling, product APIs, and agent communication.

By 2023, the problem was already visible: KV cache was huge, dynamic, fragmented, and duplicated, limiting batching. PagedAttention addressed this with OS-like paging; H2O used heavy-hitter retention; Prompt Cache introduced modular attention-state reuse; CacheGen treated KV as a compressed network payload [1][23][3][4].

In 2024, the boundary moved outward. Splitwise characterized prefill as compute-intensive and decode as memory-intensive; DistServe made this a goodput-optimized service boundary; InfiniGen showed that once KV is offloaded, the important question is selective retrieval rather than full reload; CacheBlend showed RAG chunks can be reused even when they are not prefixes if selective recomputation repairs missing cross-attention [5][6][7][11].

In 2025, the field became production-shaped. Mooncake reported a KVCache-centric platform for Kimi, using underexploited CPU, DRAM, SSD, NIC, and GPU resources and deployed at large scale [8]. KVCache Cache in the Wild and ServeGen anchored policy discussion in production traces [10][37]. Industrial stacks exposed KV transfer, tiering, and routing as product features: Mooncake Transfer Engine, Dynamo/NIXL, LMCache, vLLM integrations, and llm-d cache-aware routing [40][41][12][42].

By late 2025 and 2026, the frontier shifted again: KV is not just cached or transferred; it can be shared across related prompts, across model variants, or among agents. DroidSpeak targets fine-tuned model variants with the same architecture; KVCOMM targets overlapping multi-agent contexts under offset variance; KVComm and Q-KVComm explore KV as an inter-LLM communication medium; TokenDance exploits synchronized All-Gather-like agent rounds [17][18][19][21][20].

3. Scope: what this report considers and what it leaves out

In scope • KV cache serving: layout, placement, movement, tiering, routing, offloading, prefill/decode handoff, scheduling, product APIs. • KV reuse/sharing: exact prefix reuse, modular/non-prefix reuse, semantic reuse, cross-model reuse, multi-agent KV communication. • Representation pressure: quantization, transform coding, eviction, retention, semantic chunks, block fusion. • Adjacent model-side forces if they materially alter KV footprint or access pattern: MLA, hybrid attention, sparse prefill.	Out of scope by default • Generic transformer acceleration with no KV-state consequence. • Response-level semantic caching that returns old answers without reusing KV or model state. • Broad agent frameworks unless scheduling, sharing, or communication depends on KV state. • Weight quantization, MoE serving, speculative decoding, or compiler work unless it changes KV placement/reuse economics.
---	---

This scope follows the uploaded project intent: understand how systems place, move, compress, evict, or reuse KV state, and distinguish tensor/block mechanisms from serving-system mechanisms [P1].

4. Architectural overview: KV as a state plane

Application / Agent Runtime	declares stable segments, workflow graph, tool-call stalls, privacy labels
KV-Aware Router / Scheduler	cache affinity, load balance, TTFT/TPOT/SLO, admission, prefetch/offload
Compatibility / Metadata Plane	model id, tokenizer, layer layout, RoPE/position, dtype, quantization, tenant, TTL
Reuse and Correction Engine	prefix hit, modular chunk reuse, offset correction, semantic match, selective recompute
KV Data Plane	HBM hot tier, CPU DRAM warm tier, SSD/NVMe cold tier, remote store, RDMA/TCP/NIXL/NVMe-oF
Inference Engines	vLLM, SGLang, TensorRT-LLM, Dynamo, model-specific kernels

Figure 2. Target architecture. The emerging stack separates application structure, control decisions, compatibility metadata, correction/reuse, the KV data plane, and inference engines. Current systems each cover a subset; no single project covers the whole plane yet.

The most useful way to read the field is by asking: **which interface does this work move?** PagedAttention moves the local allocation interface; CacheGen moves the network-serialization interface; InfiniGen/IceCache/LouisKV move the retrieval boundary; Splitwise/DistServe/PrfaaS move the prefill/decode execution boundary; Mooncake moves the platform boundary; LMCache moves KV outside the engine; DroidSpeak/KVCOMM/KVComm move the compatibility and communication boundary [1][4][7][31][32][5][6][35][8][12][17][18][19].

Layer	Main interface	Representative works	Failure mode
Local substrate	Allocation, addressing, sharing within one engine	vLLM/PagedAttention, SGLang/RadixAttention, Prompt Cache	Fragmentation, duplicate KV, poor batching
Representation	What state remains, at what fidelity	H2O, KIVI, Ada-KV, KeyDiff, TRIM-KV, PolarQuant, KVTC, ChunkKV, Joint Encoding	Quality loss or overhead outweighing memory savings
Movement/offload	What to move, when, and through which tier	CacheGen, InfiniGen, HCache, IceCache, LouisKV, HiFC, SYMPHONY	Transfer latency or wrong retrieval defeats the benefit
Phase boundary	KV produced by prefill, consumed by decode	Splitwise, DistServe, Mooncake, PrfaaS, Dynamo, llm-d	Network overhead, queueing instability, placement mismatch
Control plane	Routing, scheduling, admission, load/cache tradeoff	Mooncake scheduler, DualMap, JITServe, ServeGen, llm-d	Hotspots, destroyed cache locality, SLO misses
Reuse contract	When another execution's KV is valid enough	Prompt Cache, CacheBlend, KVShare, SemShareKV, DroidSpeak, KVCOMM	Position mismatch, semantic drift, model-version mismatch
Workflow/protocol	How agents or models exchange state	KVFlow, Tokencake, Autellix/Agentix, KVComm, Q-KVComm, TokenDance	Debuggability, privacy, brittle architecture compatibility
Governance	Who may reuse which KV and why	SCX, commercial prompt-cache isolation policies	Leakage, tenant collision, invalid reuse provenance

5. Evolution: from cache optimization to serving state

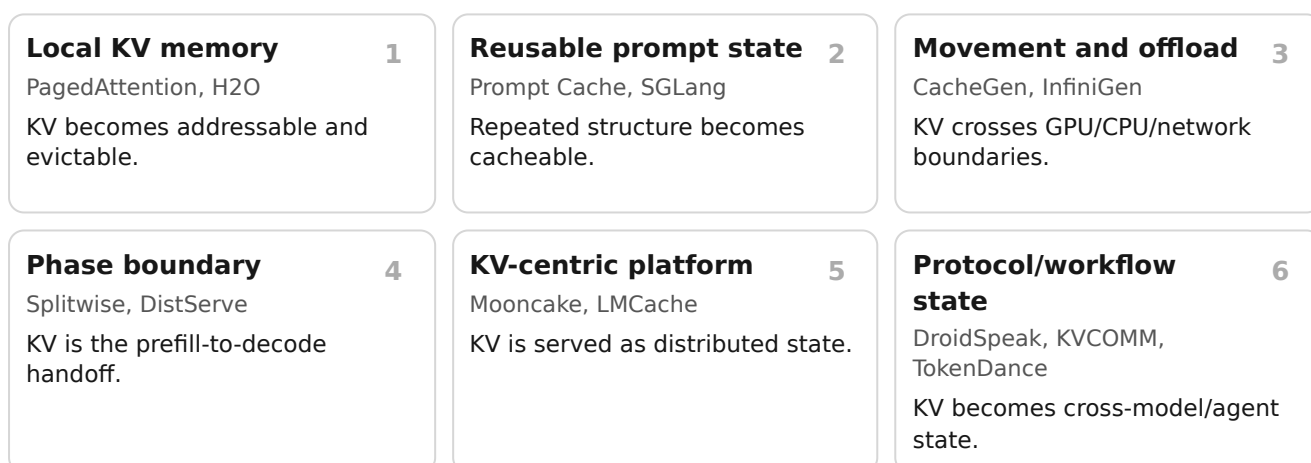


Figure 3. The boundary of KV management keeps expanding. Each stage keeps earlier stages; the field is becoming more layered, not replacing one trick with another.

5.1 Driving force: model-side KV cost

KV grows with sequence length, batch size, and number of active requests. This makes HBM footprint and memory bandwidth a first-order serving cost. Model architecture is now directly attacking the same bottleneck: DeepSeek-V2's MLA reports a 93.3% KV-cache reduction relative to DeepSeek 67B, while Kimi Linear reports up to 75% KV-cache reduction and up to 6x decoding throughput at 1M context [47][48]. Sparse prefill methods such as MInference show another side of the same force: long-context prefill is too expensive to keep treating as dense full attention everywhere [49].

5.2 Driving force: phase asymmetry

Prefill and decode are not the same workload. Splitwise frames prompt computation as compute-intensive and token generation as memory-intensive, motivating phase-specific hardware placement. DistServe turns that observation into TTFT/TPOT-aware service decomposition and

bandwidth-aware placement [5][6]. This is where KV becomes an inter-stage handoff object rather than just a tensor inside the engine.

5.3 Driving force: repeated but non-identical context

RAG, long tools, system prompts, schemas, memory, and agent traces create repeated context. But the repetition is rarely a perfect prefix. CacheBlend, EPIC, Cache-Craft, TurboRAG, and KVLink show that RAG contexts require non-prefix or position-independent cache reuse, selective recomputation, and sometimes training or mask/position changes [11][50][51][52][53]. Multi-agent systems add offset variance and synchronization patterns, which KVCOMM and TokenDance exploit explicitly [18][20].

5.4 Driving force: economics and productization

Cloud providers now expose cached tokens and context caches in public APIs. OpenAI reports cached tokens in usage fields and applies automatic prompt caching for sufficiently long repeated prefixes; Gemini supports implicit and explicit context caching; Bedrock and Azure expose prompt caching to reduce latency and input-token costs or to control retention/isolation [43][44][45][46]. This is a product signal: cached input is becoming a priced resource class, not only a hidden serving optimization.

5.5 Driving force: workload realism and schedulability

Production traces are skewed, bursty, and category-dependent. KVCache Cache in the Wild studies reuse at a large cloud provider and shows why cache policy must be workload-aware. ServeGen studies billions of production requests and exposes burstiness, shifting length distributions, multimodal composition, and reasoning-model length structure [10][37]. These findings constrain every proposed cache policy: a technique that only works under uniform synthetic prefix reuse is not a production architecture.

6. Component deep dive I: local substrate and exact reuse

PagedAttention/vLLM is the baseline because it changes KV from a monolithic tensor footprint into page-managed serving state. Its contribution is not merely speed: it gives later systems an allocatable unit for prefix reuse, block sharing, and memory accounting [1].

SGLang/RadixAttention extends reuse from identical prompts to structured language-model programs. Complex applications contain control flow, repeated system prompts, few-shot examples, retrieval chunks, and multi-call patterns. A radix tree of KV blocks is the natural runtime structure for this regime [2].

Prompt Cache adds a user/schema-facing idea: explicitly mark reusable prompt modules such as system messages, templates, and documents. This is important for future agent systems because cacheability is partly an application-level property, not only a runtime discovery problem [3].

System	Boundary moved	Why foundational	Limitation
vLLM/PagedAttention	Local KV allocation	KV becomes paged, shareable, and less fragmented.	Mostly local; distributed cache affinity remains outside the base abstraction.
SGLang/RadixAttention	Program-structured prefix reuse	Works with LM programs and repeated generation calls.	Reuse is still strongest when prefixes/segments align structurally.
Prompt Cache	Application-declared reusable modules	Exposes reusable prompt segments as a schema.	Requires discipline in prompt construction and positional correctness.

7. Component deep dive II: serving platforms and prefill/decode disaggregation

The phase-disaggregation line is the cleanest macro-architecture lineage. **Splitwise** identifies hardware specialization. **DistServe** makes phase separation SLO-aware and bandwidth-aware. **Mooncake** adds a production KV-centric platform with global cache, multi-tier resources, and KV-aware scheduling. **Prefill-as-a-Service** asks how far the boundary can move when hybrid-attention models reduce KV size enough to make cross-cluster or cross-datacenter transfer plausible [5][6][8][35].

Work/system	Primary claim	Network/storage assumption	Best comparison axis
Splitwise	Prefill and decode should be assigned to phase-suitable hardware.	Fast cluster backplane for state transfer.	Throughput, cost, power under phase split.
DistServe	Disaggregate prefill and decode to maximize goodput under TTFT and TPOT.	Cluster bandwidth matters in placement.	SLO-constrained goodput per GPU.
Mooncake	KVCache-centric production platform for Kimi with disaggregated cache and scheduler.	GPU cluster plus CPU/DRAM/SSD/NIC resources.	Effective request capacity under real traces and SLOs.
PrfaaS	Remote prefill can be selectively externalized when KV footprint shrinks.	Commodity Ethernet or loosely coupled clusters; bandwidth-aware policy required.	Cross-boundary elasticity and P90 TTFT.
Dynamo/NIXL	Efficient disaggregated serving depends on nonblocking direct KV transfer.	NIXL direct VRAM-to-VRAM transfer between prefill and decode engines.	Industrial data-plane viability.
llm-d	Distributed serving needs precise KV-cache-aware routing.	vLLM pods publish cache events; scheduler scores resident-block fraction.	Kubernetes-native cache locality and load routing.

The strategic question is no longer whether prefill/decode disaggregation is useful. It is **where the boundary belongs**: same GPU, same node, rack-scale fabric, cluster, region, or cross-datacenter. The answer depends on model-side KV footprint, workload skew, cache hit rate, network bandwidth variability, and SLO strictness.

8. Component deep dive III: movement, restoration, tiering, and retrieval

Once KV leaves GPU HBM, capacity expansion is not enough. The system must decide what to preserve, what to reconstruct, what to prefetch, and which physical path to use. CacheGen compresses KV for streaming; HCache restores state from intermediate activations; InfiniGen fetches only predicted essential entries; IceCache clusters semantic tokens for better CPU-GPU retrieval; LouisKV makes retrieval trigger/granularity workload-aware; HiFC makes flash/SSD a serious KV tier; SYMPHONY decouples compute from KV storage in multi-turn workloads [4][9][7][31][32][33][54].

Work	Retrieval/materialization unit	Trigger/policy	Productization caution
CacheGen	Encoded KV bitstream	Adapt compression to bandwidth.	Encoder/decoder overhead and integration with engine block layout.
HCache	Intermediate activation state	Restore after eviction without full recompute or raw KV reload.	New state representation must be managed alongside KV.
InfiniGen	Predicted important KV entries	Minimal rehearsal to identify next useful working set.	Prediction quality and orchestration overhead.
IceCache	Semantic clusters/pages	Hierarchical dynamic indexing and CPU-GPU transfer.	Cluster/index maintenance under live serving.
LouisKV	Different input/output retrieval units	Semantic-boundary trigger; temporal locality.	Must classify workload geometry: long-input, long-output, or both.
HiFC	Flash-backed KV blocks	Swap via flash-aware block mapping and direct path.	Endurance, sequentiality, and latency stability.
SYMPHONY	Disaggregated KV storage	Migrate KV off the critical serving path using multi-turn hints.	Scheduler and storage layer must co-design ownership and migration.

9. Component deep dive IV: reuse, sharing, and communication

Reuse moves the question from "how do I fit KV?" to "when can stored intermediate state substitute for computation?" The boundary expands in steps: exact prefix, modular segments, non-prefix chunks, semantic similarity, cross-model compatibility, cross-context agents, and finally KV as an inter-model communication medium.

Class	Representative works	Reuse boundary	Required correction
Exact prefix / radix	vLLM, SGLang	Same token prefix or structured prefix tree.	None if token sequence and positions match.
Modular / non-prefix RAG	Prompt Cache, CacheBlend, EPIC, Cache-Craft, TurboRAG, KVLink	Repeated document/template chunks at arbitrary locations.	Position adjustment, cross-attention repair, selective recompute, masks, or special tokens.
External service reuse	LMCache, Mooncake Store	Reusable text across engines, queries, GPU/CPU/disk/S3 tiers.	Engine connector, KV metadata, block mapping, transfer path.
Approximate semantic reuse	KVShare, SemShareKV	Semantically similar prompts or token matches.	Quality gate, semantic alignment, RoPE/position handling, fallback recompute.
Cross-model reuse	DroidSpeak	Fine-tuned model variants with same architecture.	Selective layer recomputation and pipelined load/recompute.
Cross-context multi-agent reuse	KVCOMM	Overlapping shared contexts under different prefixes.	Anchor-based online offset correction.
KV communication	KVComm, Q-KVComm, TokenDance	Agents or LLMs communicate selected/compressed KV or collective shared blocks.	Layer selection, compression, calibration, sparse diffs, privacy and audit policy.

The key distinction is between **serving reuse** and **communication reuse**. Serving reuse tries to avoid redundant prefill while preserving output behavior. Communication reuse asks whether the hidden representation itself is a better message than text. The second is more speculative because it is harder to debug, audit, secure, and standardize.

10. Component deep dive V: compression, quantization, eviction, and retention

Compression and retention are often flattened into one category, but they intervene at different lifecycle points. Eviction keeps less live state; quantization/coding keeps state with fewer bits;

semantic compression changes the unit of preservation; joint encoding exploits redundancy across blocks or requests.

Subfamily	Works	Unit	What to compare
Heavy-hitter / heuristic eviction	H2O, KeyDiff	Tokens, keys, recent/heavy-hitter state	Quality under fixed budget, overhead, FlashAttention compatibility.
Adaptive / learned retention	Ada-KV, TRIM-KV	Head budgets or learned token retention gates	Low-memory robustness and cross-task generality.
Low-bit quantization	KIVI, PolarQuant	KV values/keys, outlier-sensitive key dimensions	Bits, memory reduction, accuracy, decode overhead.
Transform/coding compression	KVTC	Feature-decorrelated compressed bitstream	Cold/reusable storage ratio vs encode/decode cost.
Semantic unit compression	ChunkKV	Semantic chunks rather than isolated tokens	Semantic integrity under aggressive compression.
Cross-block redundancy	Joint Encoding	Similar KV blocks across requests/chunks	Concurrency scaling and compatibility with standard layouts.

These methods are composable with, not alternatives to, disaggregated serving. A remote KV store still needs compression. A cache-aware router still needs eviction. A semantic-reuse system still needs correction and confidence thresholds. The field is converging on layered state management.

11. Component deep dive VI: routing, workload, security, and product economics

When cache locality matters, the router becomes a KV control plane. DualMap directly addresses the tension between cache affinity and load balancing with two candidate mappings and SLO-aware switching [36]. Llm-d's precise prefix-cache routing indexes per-pod vLLM KV-cache events and scores candidate pods by resident prefix fraction [42]. Mooncake makes global cache and scheduling part of the platform [8]. JITServe adds a complementary lesson: real serving has diverse SLOs and imprecise request information, so overallocating to one request can waste useful capacity [38].

Workload papers matter because cache policies are workload-shaped. KVCache Cache in the Wild provides direct production evidence on reuse and policy behavior, while ServeGen provides workload generation grounded in production traces across language, multimodal, and reasoning models [10] [37]. Security becomes unavoidable once KV is externalized or shared: SCX treats encoded KV as confidential serving state rather than ordinary cache bytes [34].

Axis	Emerging production requirement	Relevant works/docs
Routing	Route to cache without creating hotspots.	DualMap, Llm-d, Mooncake [36][42][8]
Scheduling	Optimize goodput under SLOs and uncertain work.	DistServe, JITServe, ServeGen [6][38][37]
State pricing	Expose cached-input economics to developers.	OpenAI, Gemini, Bedrock, Azure [43][44][45] [46]
Security	Protect externalized or shared intermediate state.	SCX, tenant-scoped prompt caches [34][46]
Agent programs	Schedule workflows, not isolated requests.	KVFlow, Tokencake, Autellix/Agentix [13][14] [39]

12. Future blueprint: a KV-native serving platform

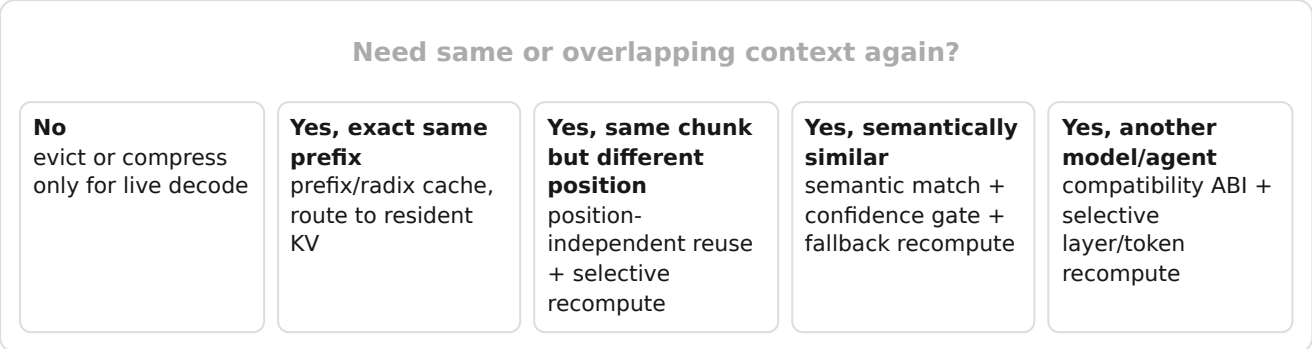


Figure 4. The product decision is a break-even problem over correctness, cost, transfer, and future reuse. The system needs metadata and guardrails to choose safely.

A future KV-native serving stack will likely standardize a **KV object ABI**. At minimum, a KV block must carry model id, architecture id, tokenizer id, layer/head layout, block size, dtype, quantization/compression format, RoPE parameters and offset, cache position, segment identity, tenant/security label, TTL, provenance, and approximate-reuse confidence. Without this, cross-engine reuse, cross-model reuse, and agent communication will remain brittle.

The agent runtime should emit more than prompts. It should emit a **cache plan**: stable segments, shared/private boundaries, tool-call stalls, expected future reuse, synchronization rounds, and approximation tolerance. KVFlow, Tokencake, Autellix/Agentix, KVCOMM, and TokenDance all point to this runtime-serving contract [13][14][39][18][20].

Future movement	Confidence	Reason	Risk
KV ABI and metadata standardization	High	Multiple engines and external KV stores need compatible movement and reuse.	Model vendors may diverge; RoPE/hybrid attention layouts complicate standardization.
Prefill-as-a-Service for hybrid-attention models	Medium-high	Model-side KV shrinkage changes cross-cluster break-even.	Dense models and variable networks keep many deployments close to RDMA/NVLink domains.
Agent cache compilers	Medium-high	Agent workflows expose structure that request-local caches cannot infer.	Framework fragmentation and privacy constraints.
Approximate semantic reuse	Medium	Potential hit-rate expansion is large.	Quality/safety gating and rollback-to-recompute are necessary.
KV communication	Medium	Strong for short-lived same-architecture cooperation.	Architecture dependence, auditability, and security.
Security/governance as first-class layer	High	Externalized KV contains sensitive context and hidden representations.	May reduce cross-tenant deduplication and reuse opportunities.

13. People and institutions: who is shaping the paradigm shift?

This is not a vanity leaderboard. The useful distinction is contribution style: who defines boundaries, who builds model-native state systems, who operationalizes platforms, who pushes compression mechanisms, and who productizes cached tokens.

Institution / cluster	Most visible contribution style	Representative works or systems	Read as
UC Berkeley / LMSYS / Sky-style systems line	Foundational runtime substrate and program-aware serving	vLLM/PagedAttention, SGLang, Autellix/Agentix, kvcached-adjacent work [1][2][39]	Early substrate and execution-runtime definers.
University of Chicago model-native-state line	KV as transferable/reusable system state	CacheGen, CacheBlend, LMCache, DroidSpeak [4][11][12][17]	The clearest sustained program on KV as model-native infrastructure.
Microsoft Research / Azure / collaborators	Phase specialization and efficient AI systems; RAG KV reuse	Splitwise, CacheBlend-related system work, MInference [5][11][49]	Boundary-shifting industrial systems research.
Peking University / UCSD / StepFun line	SLO-aware phase disaggregation and distributed serving	DistServe; related workload/routing lines via PKU/Huawei/industry collaborations [6][32][50]	Strong academic center for phase-aware serving architecture.
Moonshot AI + Tsinghua	Production KV-centric platform and cross-cluster ambitions	Mooncake, Mooncake Transfer Engine/Store, Prefill-as-a-Service [8][40][35]	High-signal production platform builder.
NVIDIA ecosystem	Vendor-grade transfer and compression/data plane	Dynamo/NIXL, KVTC, TensorRT-LLM-adjacent serving connectors [41][28]	Accelerator/network software stack setter.
Alibaba / Shanghai Jiao Tong / cloud providers	Production workload grounding	ServeGen, KVCache Cache in the Wild [37][10]	Reality checks for workload, burstiness, and cache policy.
OpenAI / Google / AWS / Azure	Cached-token productization	Prompt/context caching APIs and cached-token accounting [43][44][45][46]	Market signal that stateful token serving is priced and exposed.
Agent/KV communication groups	KV as communication/protocol object	KVCOMM, KVComm, Q-KVComm, TokenDance [18][19][21][20]	Forward-looking signal for multi-agent infra.

The strongest recurring author/institution line in this scoped corpus is the UChicago-centered model-native-state line: CacheGen -> CacheBlend -> LMCache -> DroidSpeak. The strongest production-platform line is Moonshot AI + Tsinghua: Mooncake -> Transfer Engine/Store -> Prefill-as-a-Service. The strongest phase-disaggregation academic anchor is DistServe. The strongest product signal is that cached tokens now appear in public API economics.

14. Final outlook

Bottom line. The next major systems primitive is unlikely to be just a faster attention kernel. It is a KV-state service with metadata, compatibility, security, routing, transfer, and workflow APIs.

Three practical implications follow.

1. **Serving systems must expose KV locality.** A naive load balancer destroys prefix-cache gains in distributed deployments. Future routers need near-real-time KV indexes, load estimates, and SLO objectives.
2. **Agent frameworks must compile for cacheability.** Stable prompts, tool schemas, shared documents, memory summaries, and synchronization rounds should become declared segments with reuse hints and privacy labels.
3. **Reuse needs guardrails.** Exact prefix reuse is safe; approximate semantic, cross-model, and agent-level reuse require compatibility metadata, quality confidence, fallback recomputation, and tenant isolation.

The field is moving from local KV memory efficiency to KV-aware serving infrastructure. For multi-agent systems, the decisive transition is from "agents exchange only text" to "agents exchange text, summaries, KV diffs, and selectively reusable hidden state under explicit compatibility and governance constraints."

Appendix A: important works by architectural axis

Axis	Foundational anchors	Forward signals
Local substrate	vLLM/PagedAttention, SGLang, Prompt Cache	Hybrid KV managers, kvcached-style elastic KV
Transport/restoration	CacheGen, HCache, LMCache, Mooncake TE	NIXL, GDS, NVMe-oF, CXL/shared-memory paths
Phase disaggregation	Splitwise, DistServe, Mooncake	PrfaaS, BanaServe-like balancing, xPyD configurations
Retrieval/offload	InfiniGen, IceCache, LouisKV, HiFC, SYMPHONY	Flash-native KV tiers and compute-memory disaggregation
RAG and non-prefix reuse	CacheBlend, EPIC, Cache-Craft, TurboRAG, KVLink	Position-independent caching and chunk-cache systems
Semantic/cross-model reuse	KVShare, SemShareKV, DroidSpeak	Cross-family compatibility and reusable intermediate-state registries
Multi-agent communication	KVCOMM, KVComm, Q-KVComm, TokenDance	KV collective primitives, agent cache compilers
Compression/retention	H2O, KIVI, Ada-KV, KeyDiff, TRIM-KV, PolarQuant, KVTC, ChunkKV, Joint Encoding	Hybrid policies combining retention, coding, and global redundancy
Control/workload/security	DualMap, JITServe, ServeGen, KVCache in the Wild, SCX	Cache-aware SLA controllers and secure KV governance

References

- [P0] Project resource: All Paper Analysis. *Internal project aggregation reviewed before this report.* [../data/all-paper-analysis.md](#)
- [P1] Project resource: All Project Analysis. *Internal synthesis and scope notes reviewed before this report.* [../data/all-project-analysis.md](#)
- [1] Kwon et al., Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP 2023 / vLLM.* <https://arxiv.org/abs/2309.06180>
- [2] Zheng et al., SGLang: Efficient Execution of Structured Language Model Programs. *RadixAttention and language/runtime co-design.* <https://arxiv.org/abs/2312.07104>
- [3] Gim et al., Prompt Cache: Modular Attention Reuse for Low-Latency Inference. *MLSys 2024.* <https://arxiv.org/abs/2311.04934>
- [4] Liu et al., CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. *SIGCOMM 2024.* <https://arxiv.org/abs/2310.07240>
- [5] Patel et al., Splitwise: Efficient Generative LLM Inference Using Phase Splitting. *ISCA 2024.* <https://arxiv.org/abs/2311.18677>
- [6] Zhong et al., DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving. *OSDI 2024.* <https://arxiv.org/abs/2401.09670>
- [7] Lee et al., InfiniGen: Efficient Generative Inference of LLMs with Dynamic KV Cache Management. *OSDI 2024.* <https://arxiv.org/abs/2406.19707>
- [8] Qin et al., Mooncake: Trading More Storage for Less Computation - A KVCache-centric Architecture for Serving LLM Chatbot. *FAST 2025.* <https://arxiv.org/abs/2407.00079>
- [9] Gao et al., Fast State Restoration in LLM Serving with HCache. *EuroSys 2025.* <https://arxiv.org/abs/2410.05004>
- [10] Wang et al., KVCache Cache in the Wild. *USENIX ATC 2025 production characterization.* <https://arxiv.org/abs/2506.02634>
- [11] Yao et al., CacheBlend: Fast LLM Serving for RAG with Cached Knowledge Fusion. *EuroSys 2025.* <https://arxiv.org/abs/2405.16444>
- [12] LMCache project and paper: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *External KV layer across engines and queries.* <https://arxiv.org/abs/2510.09665>
- [13] Pan et al., KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. *Workflow-aware KV cache management.* <https://arxiv.org/abs/2507.07400>
- [14] Tokencake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. *Agent/tool-call aware KV serving.* <https://arxiv.org/abs/2510.18586>
- [15] Yang et al., KVShare: Semantic-Aware Key-Value Cache Sharing for Efficient LLM Inference. *Approximate semantic KV reuse.* <https://arxiv.org/abs/2503.16525>
- [16] Zhao and Mastorakis, SemShareKV. *Token-level LSH matching and semantic KV sharing.* <https://arxiv.org/abs/2509.24832>
- [17] Liu et al., DroidSpeak: KV Cache Sharing Across Fine-tuned Model Variants. *NSDI 2026.* <https://www.usenix.org/system/files/nsdi26-liu-yuhan.pdf>
- [18] Ye et al., KVCOMM: Online Cross-context KV-cache Sharing for Efficient LLM Communication. *NeurIPS 2025.* <https://arxiv.org/abs/2510.12872>
- [19] Shi et al., KVComm: Enabling Efficient LLM Communication through Selective KV Sharing. *ICLR 2026 OpenReview.* <https://openreview.net/forum?id=F7rUng23nw>
- [20] Bian et al., TokenDance: Scaling Multi-Agent LLM Serving via Collective KV Cache Sharing. *arXiv 2026.* <https://arxiv.org/abs/2604.03143>
- [21] Kriuk and Ng, Q-KVComm: Efficient Multi-Agent Communication via Adaptive KV Cache Compression. *arXiv 2025.* <https://arxiv.org/abs/2512.17914>
- [22] Liu et al., KIVI: A Tuning-Free Asymmetric 2-bit Quantization for KV Cache. *ICML 2024.* <https://arxiv.org/abs/2402.02750>
- [23] Zhang et al., H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs. *NeurIPS 2023.* <https://arxiv.org/abs/2306.14048>
- [24] Feng et al., Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation. *NeurIPS 2025.* <https://arxiv.org/abs/2407.11550>
- [25] Park et al., KeyDiff: Key Similarity-Based KV Cache Eviction. *NeurIPS 2025.* <https://arxiv.org/abs/2504.15364>
- [26] Bui et al., Cache What Lasts: Token Retention for Memory-Bounded KV Cache in LLMs. *TRIM-KV, ICLR 2026 OpenReview/arXiv.* <https://arxiv.org/abs/2512.03324>

- [27] Wu et al., PolarQuant: Leveraging Polar Transformation for Key Cache Quantization. *NeurIPS 2025*.. <https://openreview.net/forum?id=JCTTLKEBza>
- [28] KV Cache Transform Coding for Compact Storage in LLM Inference. *ICLR 2026 OpenReview*.. <https://openreview.net/forum?id=aNVKROYpLB>
- [29] Liu et al., ChunkKV: Semantic-Preserving KV Cache Compression. *NeurIPS 2025*.. <https://arxiv.org/abs/2502.00299>
- [30] Kampeas and Haleva, Joint Encoding of KV-Cache Blocks for Scalable LLM Serving. *arXiv 2026*.. <https://arxiv.org/abs/2601.03067>
- [31] Mao et al., IceCache: Memory-efficient KV-cache Management for Long-Sequence LLMs. *ICLR 2026 OpenReview / arXiv*.. <https://openreview.net/forum?id=yHxSKM9kdr>
- [32] Wu et al., LouisKV: Efficient KV Cache Retrieval for Long Input-Output Sequences. *ICLR 2026 OpenReview/arXiv*.. <https://arxiv.org/abs/2510.11292>
- [33] HiFC: High-efficiency Flash-based KV Cache Swapping for Scaling LLM Inference. *NeurIPS 2025*.. <https://openreview.net/forum?id=onhjdWCxZY>
- [34] Yuan et al., SCX: Stateless KV-Cache Encoding for Cloud-Scale Confidential Transformer Serving. *SIGCOMM 2025*.. <https://dl.acm.org/doi/10.1145/3718958.3750509>
- [35] Qin et al., Prefill-as-a-Service: KVCache of Next-Generation Models Could Go Cross-Datcenter. *arXiv 2026*.. <https://arxiv.org/abs/2604.15039>
- [36] Yuan et al., DualMap: Enabling Both Cache Affinity and Load Balancing for Distributed LLM Serving. *arXiv 2026*.. <https://arxiv.org/abs/2602.06502>
- [37] Xiang et al., ServeGen: Workload Characterization and Generation of LLM Serving in Production. *NSDI 2026 / Alibaba traces*.. <https://arxiv.org/abs/2505.09999>
- [38] Zhang et al., JITServe: SLO-aware LLM Serving with Imprecise Request Information. *NSDI 2026*.. <https://www.usenix.org/conference/nsdi26/presentation/zhang-wei>
- [39] Luo et al., Autellix / Agentix: An Efficient Serving Engine for LLM Agents as General Programs. *NSDI 2026*.. <https://arxiv.org/abs/2502.13965>
- [40] Mooncake Transfer Engine documentation. *TCP, RDMA, CXL/shared memory, NVMe-oF transfer paths*.. <https://kvcache-ai.github.io/Mooncake/design/transfer-engine/index.html>
- [41] NVIDIA Dynamo disaggregated serving documentation. *NIXL-based KV transfer*.. <https://docs.nvidia.com/dynamo/design-docs/disaggregated-serving>
- [42] Llm-d precise prefix-cache aware routing documentation. *KV-cache event indexing and routing*.. <https://llm-d.ai/docs/guide/Installation/precise-prefix-cache-aware>
- [43] OpenAI API prompt caching documentation. *Cached token reporting and behavior*.. <https://developers.openai.com/api/docs/guides/prompt-caching>
- [44] Google Gemini API context caching documentation. *Implicit and explicit context caching*.. <https://ai.google.dev/gemini-api/docs/caching>
- [45] Amazon Bedrock prompt caching documentation. *Prompt caching for lower latency and input token cost*.. <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html>
- [46] Azure OpenAI prompt caching documentation. *Prompt cache retention and isolation*.. <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/prompt-caching>
- [47] DeepSeek-V2: A Strong, Economical, and Efficient MoE Language Model. *MLA and KV cache reduction*.. <https://arxiv.org/abs/2405.04434>
- [48] Kimi Linear: An Expressive, Efficient Attention Architecture. *Hybrid attention and KV footprint reduction*.. <https://arxiv.org/abs/2510.26692>
- [49] Jiang et al., MInference: Million-Tokens Prompt Inference for Long-context LLMs. *Dynamic sparse prefill*.. <https://arxiv.org/abs/2407.02490>
- [50] EPIC: Efficient Position-Independent Context Caching for Serving LLMs. *ICML 2025 / position-independent caching*.. <https://arxiv.org/abs/2410.15332>
- [51] Cache-Craft: Managing Chunk-Caches for Efficient RAG. *SIGMOD 2025*.. <https://arxiv.org/abs/2502.15734>
- [52] TurboRAG: Accelerating RAG with Precomputed KV Caches for Chunked Text. *EMNLP 2025*.. <https://aclanthology.org/2025.emnlp-main.334/>
- [53] Yang et al., KVLink: Accelerating LLMs via Efficient KV Cache Reuse. *NeurIPS 2025*.. <https://arxiv.org/abs/2502.16002>
- [54] SYMPHONY: Enabling Compute-Memory Disaggregation in LLM Serving Systems. *NSDI 2026*.. <https://www.usenix.org/conference/nsdi26/presentation/agarwal>